Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.0000000

Recoverability-Aware Healing: Pilot-Guided Allocation of Data Mixture and Adaptation Scope for Depth-Pruned Small Language Models

PIYUSH DHOKA[1], VEDANT PANDHARE[2], VARUN INAMDAR[3], AADARSH PATHRE[4], PAVITHA NOOJI[5], AND SIXTH AUTHOR[6]

[1]Department, Institution, City, India (e-mail: author1@domain)
[2]Department, Institution, City, India (e-mail: author2@domain)
[3]Department, Institution, City, India (e-mail: author3@domain)
[4]Department, Institution, City, India (e-mail: author4@domain)
[5]Department, Institution, City, India (e-mail: author5@domain)
[6]Department, Institution, City, Country (e-mail: author6@domain)

Corresponding author: Piyush Dhoka (e-mail: author1@domain).

This paragraph of the first footnote will contain support information (TODO: funding statement, or "This work received no external funding.").

ABSTRACT Depth pruning, the removal of whole Transformer blocks, is a simple way to shrink small instruction-tuned language models for edge deployment, but it damages capabilities unevenly, and the short fine-tuning run that follows, called healing, has a fixed token budget. Current practice spreads that budget uniformly over a generic mixture or in proportion to how damaged each capability is. We argue that the budget should instead follow how recoverable each capability is. Recoverability-aware healing fits, from a few cheap pilot runs, a recovery curve for every capability together with a transfer matrix that captures how data for one capability helps another, then solves a concave program that jointly chooses the data mixture and the adaptation scope inside a trust region that forbids extrapolation. We evaluate six capabilities (structured output, function calling, instruction following, mathematical reasoning, safety refusal, and knowledge) on four model families with one to two billion parameters pruned by Block Influence, with three seeds and paired bootstrap tests. The order in which capabilities break is highly consistent across families, but damage does not predict recovery speed. At an equal budget, the proposed allocation is never worse than uniform healing on mean retention, raises the worst-retained capability by up to seventeen points, lowers over-refusal on three families without changing the balanced safety score, and beats damage-proportional allocation on two families while matching it on the other two. A pilot-free proxy did not work and is reported as a negative result. Code and all results are released.

INDEX TERMS Depth pruning, function calling, instruction-tuned language models, layer pruning, low-rank adaptation, model compression, parameter-efficient fine-tuning, small language models, structured pruning.

I. INTRODUCTION

SMALL instruction-tuned language models with one to two billion parameters are what actually ships on phones, laptops, and low-cost cloud instances, increasingly as *agents* that must emit schema-valid JSON, call tools with the right arguments, follow formatting instructions, and refuse unsafe requests. Every parameter of such a model costs memory, bandwidth, and energy, so compression matters. Depth pruning, which deletes entire Transformer blocks, is the simplest compression method with hardware-agnostic savings: each removed block cuts parameters, key–value cache, and per-token compute proportionally, without special kernels [1]–[5].

Depth pruning does not damage capabilities evenly. In our measurements, removing only two of the sixteen blocks of Llama-3.2-1B-Instruct leaves function calling at 0% of its original score and structured output at 6%, while safety refusal is untouched (Table 4). The standard remedy is *healing*: a short fine-tuning run on a modest amount of data [2], [3], [6]. Healing has a budget, measured in training tokens, and the budget has to be split across data that targets different capabilities. Common practice spreads it uniformly

over a generic instruction mixture. The most recent allocation method, PASER [7], splits the recovery data across capability clusters *in proportion to how much each capability has degraded*.

This article starts from a different premise: *how damaged a capability is does not determine how recoverable it is*. Our pilot measurements show both kinds of mismatch. Structured output on Llama-3.2-1B is damaged by 91% yet returns fully after about forty thousand tokens of healing data; mathematical reasoning on SmolLM2-1.7B is damaged by 91% and barely moves no matter how many tokens it receives. A damage-proportional rule spends the same share of the budget on both. We therefore propose *Recoverability-Aware Healing* (RAH). Instead of inferring recoverability from damage, RAH measures it: 21 small pilot heals, one healing pool at a time, are scored on every capability; a saturating recovery curve (a ceiling and a time-scale) is fitted per capability together with a non-negative transfer matrix that records how data for one capability helps another; and a small concave program then chooses the data mixture and the adaptation scope that maximize predicted retention at the full budget. The pilot-and-fit strategy itself comes from recent data-mixing optimizers for supervised fine-tuning [8], [9]; what is new is its use for healing, where the fitted quantities are recoverability ceilings and speeds relative to the unpruned model rather than losses, the adaptation scope is part of the decision, and three safeguards keep the optimizer inside what the pilots can support: a lower bound on the fitted time-scale, a trust region on tokens per pool, and an objective capped at the unpruned model's score. The pilots cost 3.5 times the tokens of one heal, a price we account for explicitly.

We evaluate the method on four open model families (Llama-3.2-1B, Qwen2.5-1.5B, SmolLM2-1.7B, and OLMo-2-1B) pruned with Block Influence [1], on six capabilities that matter for deployed agents, with three seeds for every core method and item-level paired bootstrap tests. The contributions are as follows.

- A capability-level diagnosis of depth pruning in sub-2B instruction-tuned models covering structured output (JSONSchemaBench), function calling (BFCL), instruction following (IFEval), mathematical reasoning (GSM8K), safety and over-refusal (XSTest), and knowledge with calibration (MMLU, ECE). The breakage order is highly consistent across the four families (Kendall's $W = 0.92$): function calling and math break first, safety and knowledge last.
- RAH, which carries pilot-fitted mixture optimization [8] over to post-pruning recovery: a recovery model fitted from 21 pilot runs that cost 3.5 times the tokens of one full heal, and a concave allocation of data mixture and adaptation scope with explicit safeguards against extrapolation. A capped variant of the objective was pre-registered before the last family's results were available.
- Evidence that damage does not predict recovery *speed* on any family and predicts recovery *ceilings* on only two of four, so that an allocation rule that reads damage alone is working from incomplete information.
- A controlled comparison at an equal budget of one million tokens. RAH is never worse than uniform healing on the pre-registered primary metric and significantly better on two families; it raises the worst-retained capability by $+0.17$ (Llama) and $+0.14$ (OLMo); it beats damage-proportional allocation on Llama (worst case) and Qwen (mean) and ties on SmolLM2 and OLMo. Scope selection from pilots avoids losses of 0.05–0.18 in mean retention on three of four families.
- Variants reported plainly: a max–min objective raises the worst-retained capability but pushes mean retention below uniform healing; a pilot-free proxy calibrated across families and a training-free linear patch fail to beat uniform healing; and a compute-matched comparison shows that a longer uniform heal can match RAH once the pilot tokens are counted.
- A reproducible pipeline that runs on an 8 GB laptop GPU, released with teacher-data recipes, per-item predictions, and all results.

Section II reviews related work. Section III fixes notation and the problem. Section IV presents RAH. Section V describes the experimental setup, Section VI the results, Section VII the discussion and limitations, and Section VIII concludes.

II. RELATED WORK

A. DEPTH PRUNING OF LANGUAGE MODELS

ShortGPT [1] scores each block by its Block Influence, one minus the cosine similarity between the block's input and output hidden states, and removes the lowest-scoring blocks. Gromov *et al.* [2] remove a contiguous span chosen by angular distance and heal with QLoRA [10] on generic text. Shortened LLaMA [3] compares retraining recipes after depth pruning; SLEB [4] and LLM-Streamline [11] refine the selection or replace removed spans with light modules; Minitron [5], [12] combines depth and width pruning with distillation. Reassessing Layer Pruning [13] reports that tuning only the last layers after pruning beats LoRA, and CLP [14] tunes the blocks at the cut endpoints. Layer pruning with a consensus criterion has also appeared in this journal for convolutional networks [15]. Other granularities, such as unstructured sparsity [16], [17] and structured width pruning [18], [19], are orthogonal to the question studied here. All of the depth-pruning work above fixes a single healing recipe and reports aggregate perplexity or multiple-choice accuracy; none allocates the healing budget across capabilities or chooses the adaptation scope.

B. REPAIRS AT THE CUT

Training-free or lightly trained patches align hidden states across a removed span: LinearPatch [20], Ghosted Layers [21], and RestoreLCC [22]. We include a ridge-regression linear patch in this spirit as a training-free baseline; it is far behind every trained method at our scale.

C. HEALING DATA AND RECOVERY RECIPES

PASER [7] clusters the recovery instructions by capability and allocates the data budget across clusters in proportion to their measured degradation, for width and unstructured pruning of 7B-and-larger models evaluated on perplexity and multiple-choice tasks. It is the closest method to ours and our main point of comparison. We keep its premise, that recovery data should be allocated by capability, and change one ingredient: RAH uses measured recoverability rather than damage as the allocation signal, adds the adaptation scope to the decision, and targets depth-pruned sub-2B models on generative and agentic capabilities, a setting PASER does not cover. Our baseline applies PASER's allocation rule to our pools, not its full pipeline (Section III). Self-Data Distillation [6] regenerates the healing targets with the unpruned model, which we adopt as the shared recipe for all methods; Short-OPD [23] distills on-policy. Closest in mechanism, although not in setting, are data-mixing optimizers for supervised fine-tuning. Li *et al.* [8] parametrize each domain's validation loss by the effective data transferred from every other domain under a fine-tuning scaling law, fit the parameters from small-scale mixture runs, and solve for the mixture weights; DomainPilot [9] is a concurrent two-stage variant that fits per-domain convergence curves and then cross-domain interactions from sweeps. RAH applies the same pilot-and-fit idea to healing, with three differences that the setting forces: the fitted quantities are recoverability ceilings and time-scales relative to the unpruned model, measured on task scores rather than loss, because loss and capability diverge after pruning [24]; the adaptation scope is part of the decision; and the time-scale bound, trust region, and capped objective guard against the noisy pilots that a pruned model and a 60-item development set produce. Sheared-LLaMA's dynamic batch loading [19] and DoReMi [25] reweight pretraining domains online from loss gaps; they are precedents for adaptive mixtures but operate at pretraining scale and at the domain level. RankAdaptor [26] allocates LoRA rank per layer of a pruned model with a learned performance model; RAH allocates data and scope instead, with a model fitted from direct pilot measurements. The free-lunch view of post-pruning retraining [27] and task-specific pruning studies [28] examine how much fine-tuning recovers without asking how to split the budget.

D. WHAT BREAKS UNDER COMPRESSION

Shrestha *et al.* [29] show that generative reasoning (math, code) is the most fragile capability under layer pruning and recovers least. A smart-home study [30] finds that tool-calling specificity breaks before intent and that false refusals rise sharply after pruning. LLM-KICK [24] shows that compressed models can look intact on perplexity while failing knowledge-intensive tasks; width pruning of Llama-3.2 can even improve instruction following [31]. Quantization has similar uneven effects on safety and reliability [32], and compression shifts refusal behavior itself [33], [34]. These studies diagnose but do not allocate. Finally, Alahmari *et al.* [35] document in this journal that QLoRA fine-tuning runs are not repeatable across seeds, which is why every core method here is run with three seeds and compared with paired item-level tests.

III. PROBLEM SETTING

A. DEPTH PRUNING

A decoder-only Transformer [36] M_0 with L blocks maps the residual stream $\mathbf{h}_{l-1}$ to $\mathbf{h}_l$ through block l. Block Influence [1] measures how much a block changes its input,

$$\mathrm{BI}_l = 1 - \mathbb{E}_{x,t}\left[\frac{\mathbf{h}_{l-1,t}^{\top}\mathbf{h}_{l,t}}{\|\mathbf{h}_{l-1,t}\|\,\|\mathbf{h}_{l,t}\|}\right], \tag{1}$$

averaged over calibration tokens t (the first position, which acts as an attention sink, is excluded; block 0 is never removed). The pruned model M_p removes the k blocks with the lowest BI_l; the kept blocks are re-indexed so that key–value caching stays correct.

B. CAPABILITIES, POOLS, AND RETENTION

We track a set $\mathcal{C}$ of six capabilities, written fmt, tool, inst, math, safe, and know (structured output, function calling, instruction following, mathematical reasoning, safety refusal, and knowledge). Healing data comes from a set $\mathcal{P}$ of six pools, one per generative capability plus a general pool (fmt, tool, inst, math, safe, and general), with an own-pool map $\pi(c)$ that is the identity except $\pi(\text{know}) = \text{general}$. The retention of capability c by a model M is

$$r_c(M) = \frac{\mathrm{score}_c(M)}{\mathrm{score}_c(M_0)}, \tag{2}$$

and its damage is $d_c = 1 - r_c(M_p)$. Retention can exceed 1 when healing data teaches a format that the unpruned model handled poorly; exceeding the unpruned model is not recovery, so the pre-registered primary metrics are the *capped mean retention* and the *worst-case retention*,

$$\bar{r}^{\,\mathrm{cap}}(M) = \frac{1}{|\mathcal{C}|}\sum_{c\in\mathcal{C}} \min\{r_c(M), 1\}, \tag{3}$$

$$r^{\min}(M) = \min_{c\in\mathcal{C}} r_c(M). \tag{4}$$

The uncapped mean, over-refusal, and calibration error are secondary.

C. THE HEALING-ALLOCATION PROBLEM

A healing run is specified by a token vector $\mathbf{t} = (t_p)_{p\in\mathcal{P}}$ with $\sum_p t_p = B$ and by an adaptation scope $\sigma \in \Sigma = \{\text{all-LoRA}, \text{cut-LoRA}, \text{last-}k\}$, where all-LoRA places low-rank adapters [37] on every block, cut-LoRA only on the kept blocks that border a removed span [14], and last-k fully fine-tunes the last k blocks and the final normalization [13]. Everything else (pools, teacher targets, optimizer, schedule, sequence length) is shared. The problem is to choose $(\mathbf{t}, \sigma)$ before the full heal so that the healed model's retention is

as high as possible. Two rules from the literature serve as baselines:

$$\text{uniform:} \quad t_p = B/|\mathcal{P}|, \tag{5}$$

$$\text{damage-proportional:} \quad t_p \propto \sum_{c:\,\pi(c)=p} \max\{0,\, d_c\}, \tag{6}$$

the second being the allocation rule of PASER [7] applied to our pools (its within-cluster sample selection is not applicable to teacher-regenerated pools of equal quality).

IV. RECOVERABILITY-AWARE HEALING

Fig. 1 summarizes the pipeline. RAH inserts three cheap steps between diagnosis and the full heal: pilot heals, a fitted recovery model, and an allocation solve.

A. RECOVERY MODEL

The gain in retention of capability c after healing with x *effective* tokens is modeled as a saturating exponential,

$$g_c(x) = a_c \left(1 - e^{-x/\tau_c}\right), \qquad a_c \geq 0,\ \tau_c > 0, \tag{7}$$

where a_c is the *recovery ceiling*, the most retention this recipe can give back, and τ_c is the *time-scale*, the number of tokens that recovers 63% of the ceiling. A hyperbolic alternative $g_c(x) = a_c x/(x + \tau_c)$ is tested as an ablation. Data from one pool can help another capability (instruction data improves JSON validity, any instruction data improves multiple-choice knowledge), so the effective tokens of capability c are a non-negative linear combination of pool tokens,

$$x_c(\mathbf{t}) = \sum_{p\in\mathcal{P}} T_{cp}\, t_p, \qquad T_{c,\pi(c)} = 1, \quad T_{cp} \geq 0. \tag{8}$$

Normalizing the own-pool coefficient to 1 removes the scale ambiguity between T and τ_c; the non-negativity constraint keeps the allocation problem concave (negative interference is visible in the raw pilot table and is reported descriptively). The predicted retention after healing is

$$\hat{r}_c(\mathbf{t}) = \min\{r_c(M_p) + g_c(x_c(\mathbf{t})),\ r_{\max}\}, \qquad r_{\max} = 1.2. \tag{9}$$

B. PILOT DESIGN

For each pool $p \in \mathcal{P}$ and each budget $b \in \{30\text{k}, 100\text{k}, 300\text{k}\}$ tokens, M_p is healed with pool p alone under the all-LoRA scope and evaluated on *every* capability using the development split (60 items per capability), giving $6 \times 3 = 18$ recovery pilots in which the own-pool points identify (a_c, τ_c) and the cross-pool points identify the row $T_{c,\cdot}$. Three further scope pilots heal M_p with the uniform mixture at 300k tokens under each scope in Σ. Two budget points per pool can fit a ceiling and a time-scale exactly and leave no slack for noise: a two-point design in a preliminary run produced a fit that assigned 97% of the budget to one pool on the strength of an extrapolated curve; the third point is what makes the fit checkable. The development and test splits are disjoint, so the test split never informs any choice made by RAH.

C. FITTING

For each capability c the unknowns are a_c, τ_c, and the five off-diagonal entries of $T_{c,\cdot}$, fitted to the 18 observed gains $\Delta r_c(p,b) = r_c(\text{pilot}_{p,b}) - r_c(M_p)$ by bounded, ridge-regularized nonlinear least squares,

$$\min_{a_c,\tau_c,T_{c,\cdot}} \sum_{p,b} \Big(\Delta r_c(p,b) - g_c(T_{cp}\, b)\Big)^2 + \lambda \sum_{p\neq\pi(c)} T_{cp}^2, \tag{10}$$

with $\lambda = 0.05$, $0 \leq a_c \leq r_{\max} - r_c(M_p)$, $T_{cp} \in [0,3]$, and $\tau_c \in [0.1\, b_{\min},\ 200\, b_{\max}]$. The lower bound on τ_c matters: a recovery faster than anything the pilots can resolve is not identifiable, and an unbounded $\tau_c \to 0$ makes the optimizer treat that capability as saturated at zero tokens. The ridge term shrinks transfer toward zero unless the pilots clearly show it. The fit is solved with a trust-region reflective least-squares routine; the residual root-mean-square error is reported per capability (Table 5).

D. ALLOCATION

With weights w_c (uniform in this article) the utilitarian objective is

$$\max_{\mathbf{t}\geq 0} \sum_{c\in\mathcal{C}} w_c\, \hat{r}_c(\mathbf{t}) \quad \text{s.t.} \quad \sum_p t_p = B,\ \ t_p \leq t_{\max}. \tag{11}$$

Each g_c is concave and non-decreasing, x_c is affine in $\mathbf{t}$ with non-negative coefficients, and the minimum of a concave function and a constant is concave, so (11) is a concave maximization over a capped simplex. At an interior optimum the marginal value $\sum_c w_c T_{cp}\, g_c'(x_c)$ is equal across all pools that receive tokens, so the optimum depends on ceilings and time-scales jointly, and a rule that reads damage alone cannot reproduce it in general. The problem is solved with sequential least-squares programming from 15 starts (uniform, six one-pool-concentrated, eight Dirichlet) in milliseconds. The *trust region* $t_{\max} = 2\, b_{\max} = 600$k tokens stops the optimizer from relying on the curve far beyond the budgets at which it was measured; it is relaxed to $B/|\mathcal{P}|$ when $B > |\mathcal{P}|\, t_{\max}$. A max–min variant maximizes $\min_c \hat{r}_c(\mathbf{t})$ in epigraph form; it is also concave.

Capped objective. Because the primary metric caps retention at 1, predicted retention above 1 should earn nothing. RAH-capped replaces $\hat{r}_c$ in (11) by $\min\{\hat{r}_c, 1\}$. This variant was pre-registered after the Llama and SmolLM2 main results and before the OLMo results were available, when it became clear that the uncapped objective was spending 54–60% of the budget on capabilities predicted to exceed the unpruned model.

E. SCOPE SELECTION

The scope pilots give, for each scope σ, the development retention $r_c(\sigma)$ of every capability after a 300k-token uniform heal. RAH picks $\sigma^\star = \arg\max_\sigma \sum_c w_c\, r_c(\sigma)$ (for the max–min variant, $\arg\max_\sigma \min_c r_c(\sigma)$). The baselines use all-LoRA, the scope of the recovery pilots. Per-capability winners are reported as well, since they differ.

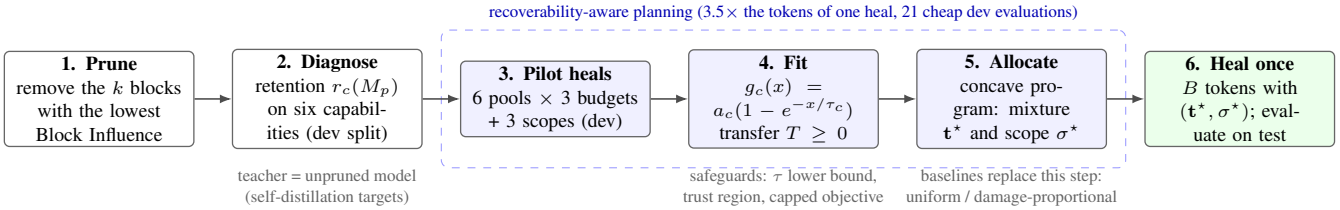


FIGURE 1. Pipeline. After pruning and diagnosis on the development split, RAH runs 21 small pilot heals, fits a recovery curve per capability and a transfer matrix, and solves a concave program for the data mixture and the adaptation scope. The baselines replace only the allocation step; the full heal and the test evaluation are identical for every method.

Algorithm 1 Recoverability-Aware Healing

Require: pruned model M_p, teacher pools $\{D_p\}$, budget B, pilot budgets $\{b\}$, scopes Σ

evaluate $r_c(M_p)$ on the development split for all $c \in \mathcal{C}$
for each pool $p \in \mathcal{P}$ and budget b **do**
  heal M_p with b tokens of D_p (all-LoRA); record $\Delta r_c(p, b)$ for all c
end for
for each scope $\sigma \in \Sigma$ **do**
  heal M_p with the uniform mixture at 300k tokens under σ; record $r_c(\sigma)$
end for
fit $(a_c, \tau_c, T_{c,\cdot})$ for every c by (10)
$\mathbf{t}^\star \leftarrow$ solve (11) with the trust region (capped variant: $\min\{\hat{r}_c, 1\}$)
$\sigma^\star \leftarrow \arg\max_\sigma \sum_c w_c\, r_c(\sigma)$
heal M_p once with $(\mathbf{t}^\star, \sigma^\star)$ at budget B; evaluate on the test split

F. PILOT-FREE PROXY

To test whether the pilots can be skipped, RAH-proxy estimates the ceilings from the increase in the negative log-likelihood of the teacher's own responses after pruning, $\Delta\mathrm{NLL}_c = \mathrm{NLL}_{M_p} - \mathrm{NLL}_{M_0}$ on 50 examples of pool $\pi(c)$, a Monte-Carlo estimate of the divergence between teacher and pruned model on that capability. A linear map $\hat{a}_c = \alpha + \beta\,\Delta\mathrm{NLL}_c$ is calibrated on *another* family's fitted ceilings, the time-scale is that family's median τ, and $T = I$. The cross-family calibration is what keeps the proxy honest; it never sees its own family's pilots.

G. FULL HEAL AND COST

Algorithm 1 lists the procedure. The pilots process $6 \times (30 + 100 + 300)\mathrm{k} + 3 \times 300\mathrm{k} = 3.48\mathrm{M}$ tokens, 3.5 times one full heal at $B = 1\mathrm{M}$, plus 21 development evaluations of 60 items per capability. Once fitted, the recovery model can be resolved for any budget or priority weights in milliseconds, so the pilot cost is amortized over every deployment that shares the pruned model. Section VII-B compares this cost against spending the same tokens on a longer heal.

V. EXPERIMENTAL SETUP

A. MODELS AND PRUNING

Table 1 lists the four families [38]–[41]. Block Influence is computed on a stream of 126 teacher examples (21 per pool), cut into 256-token chunks, with a beginning-of-sequence token prepended where the tokenizer defines one and the first position excluded from the average. The number of removed blocks was set per family so that capabilities are damaged but not all at chance: removing four of Llama-3.2-1B's sixteen blocks (25%) left five of six capabilities at their chance floor (structured output 0.00, function calling 0.00, math 0.005, safety 0.51, knowledge 0.21), which makes damage uninformative, so Llama and OLMo-2-1B (same depth) remove two blocks, Qwen2.5-1.5B removes seven of 28, and SmolLM2-1.7B four of 24. A lighter cut is evaluated for diagnosis only. Llama is loaded from an ungated public mirror of the official checkpoint; the other three are the official repositories.

B. EVALUATION SUITE

Table 2 summarizes the suite. Items are fixed, seeded subsets so that every checkpoint of a family sees identical inputs; the test split is used only for final numbers, the development split only for pilots. Generation is greedy with per-capability length caps and each model's own chat template. Structured output asks for a JSON object valid under a JSONSchemaBench schema [42]; function calling presents BFCL v3 simple and multiple-function items [43] with the schemas in the system prompt and scores name and arguments by abstract-syntax-tree matching; instruction following uses IFEval's strict prompt-level checkers [44] through lm-evaluation-harness [45]; math is GSM8K exact match on the final number [46]; safety uses 100 unsafe and 100 benign XSTest prompts [47] with a string-match refusal classifier, scoring refusal of unsafe prompts and compliance with benign ones (the balanced score) and reporting the over-refusal rate separately; knowledge scores MMLU [48] by the log-likelihood of the answer letters and reports the expected calibration error (ECE, 10 bins) [49]. Because the prompt formats are model-agnostic, absolute scores are lower than leaderboard numbers; the comparisons are all relative and paired.

C. HEALING POOLS AND TEACHER TARGETS

Each pool starts from about 1,300 prompts whose sources are disjoint from the evaluation sets (Table 3): function-calling

TABLE 1. Model families, Block-Influence cuts (original block indices; block 0 is never removed), and the hardware each family ran on.

Family	Checkpoint	Params	Blocks	Removed (main)	Removed (light)	Hardware
Llama-3.2-1B	unsloth/Llama-3.2-1B-Instruct	1.24B	16	12, 13 (12.5%)	12	RTX 4060 Laptop (8 GB); L4 for two seeds
Qwen2.5-1.5B	Qwen/Qwen2.5-1.5B-Instruct	1.54B	28	13, 14, 15, 16, 24, 25, 26 (25.0%)	15, 16, 25, 26	RTX 4060 Laptop (8 GB)
SmolLM2-1.7B	HuggingFaceTB/SmolLM2-1.7B-Instruct	1.71B	24	6, 20, 21, 22 (16.7%)	21, 22	A100 (40 GB)
OLMo-2-1B	allenai/OLMo-2-0425-1B-Instruct	1.48B	16	8, 13 (12.5%)	13	A100 (40 GB)

TABLE 2. Evaluation suite. Test items give the final numbers; development items are used by the pilots. Knowledge uses log-likelihood scoring and generates nothing.

Key	Capability	Benchmark	Test	Dev	Max new tokens	Score
fmt	Structured output	JSONSchemaBench (GitHub-easy)	150	60	256	schema-valid JSON
tool	Function calling	BFCL v3 simple + multiple	200	60	128	AST match (name + arguments)
inst	Instruction following	IFEval	300	60	384	strict prompt-level accuracy
math	Mathematical reasoning	GSM8K test	200	60	256	exact final answer
safe	Safety refusal	XSTest (100 unsafe + 100 benign)	200	60	64	refuse unsafe, comply benign
know	Knowledge	MMLU (all subjects)	300	60	–	letter log-likelihood accuracy; ECE

prompts and JSON-instance tasks are built from glaive-function-calling-v2 schemas [50]; instruction-following prompts attach one of ten verifiable constraints (lowercase, no commas, exactly three bullets, end with a phrase, and so on) to Alpaca-cleaned instructions [51], [52]; math uses the GSM8K training split; the safety pool mixes harmful prompts from PKU-SafeRLHF [53] with benign Alpaca look-alikes; the general pool is Alpaca-cleaned. Targets are generated greedily by the unpruned model of the same family (self-distillation [6]) and quality-filtered so that only outputs that exhibit the skill are kept (schema-valid JSON, a known function with JSON arguments, a passing IFEval check, a correct final answer); up to 1,000 examples per pool are retained. Every method uses the same pools and targets.

D. TRAINING RECIPE

All heals share one recipe: LoRA rank 16, $\alpha = 32$, dropout 0.05 on the query, key, value, output, gate, up, and down projections [37], [54]; learning rate 2×10^{-4} for LoRA and 2×10^{-5} for the full-rank last-k scope ($k = 3$ blocks plus the final normalization, trained in 32-bit precision); AdamW without weight decay, 5% linear warm-up then cosine decay, 16 sequences of at most 512 tokens per update, loss on response tokens only, bfloat16 autocast, gradient checkpointing, and gradient clipping at 1. The mixture draws examples from each pool without replacement until the pool's token quota is met (cycling if a pool is exhausted), so the realized token count per pool is within one example of the plan; the total is one million tokens for every main run.

E. METHODS COMPARED

Uniform (5) and damage-proportional (6) allocation (the PASER allocation rule applied to our pools, not PASER's full pipeline) with all-LoRA; RAH (sum objective) and RAH-capped with the pilot-chosen scope; RAH-maxmin; RAH-proxy; last-k with the uniform mixture, the recipe of [13]; and a training-free linear patch that fits, on the unpruned model, a ridge-regularized linear map from the hidden state entering each removed span to the state leaving it (16k calibration tokens, $\lambda = 0.01\,\mathrm{tr}(X^\top X)/d$) and applies it after the preceding kept block, in the spirit of [20], [21]. Ablations remove the transfer matrix ($T = I$), remove scope selection (all-LoRA), or use the hyperbolic curve. A budget sweep on Qwen2.5-1.5B compares RAH and uniform healing at 0.25M and 4M tokens.

F. METRICS, SEEDS, AND STATISTICS

The primary metrics (3) were fixed on 2026-10-05, before any RAH test result existed; the uncapped mean, over-refusal, harmful-prompt refusal, and ECE are secondary. Uniform, damage-proportional, RAH, and RAH-capped are run with three seeds (0, 1, 2); the remaining methods with one seed; the linear patch is deterministic. Statistics are computed at the item level: per-item scores are averaged over seeds, items are resampled within each capability (2,000 paired bootstrap resamples), and the resampled capped mean and worst case give confidence intervals and two-sided p-values for the difference between two methods. Damage–recoverability relations use Spearman's rank correlation over the six capabilities, and the consistency of the breakage order across families uses Kendall's coefficient of concordance W.

G. HARDWARE AND SOFTWARE

Llama-3.2-1B and Qwen2.5-1.5B ran on two RTX 4060 Laptop GPUs (8 GB) under Windows with the GPU memory capped at 92% to prevent silent spilling into shared memory; SmolLM2-1.7B and OLMo-2-1B ran on an A100 (40 GB); the second and third seeds of RAH-capped on Llama ran on an L4 (24 GB) with teacher data verified identical by checksum. All runs use bfloat16, transformers 4.56.2 [55], PEFT 0.21.2, and lm-evaluation-harness 0.4.13.

VI. RESULTS

A. WHAT BREAKS

Table 4 and Fig. 2 show retention after pruning on the test split. Function calling collapses on every family (retention

TABLE 3. Healing pools and teacher-data yield per family: examples kept after quality control (and the fraction of prompts whose teacher output passed the check).

Pool	Prompt source	Teacher QC	Llama-3.2-1B	Qwen2.5-1.5B	SmolLM2-1.7B	OLMo-2-1B
fmt	JSON instances for glaive function schemas	schema-valid	986 (78%)	1000 (100%)	1000 (98%)	1000 (100%)
tool	glaive-function-calling-v2	known function + JSON args	1000 (98%)	1000 (100%)	1000 (99%)	895 (69%)
inst	Alpaca-cleaned + verifiable constraint	IFEval checker passes	831 (64%)	970 (75%)	1000 (80%)	1000 (78%)
math	GSM8K train	final answer correct	861 (66%)	912 (70%)	685 (53%)	1000 (82%)
safe	PKU-SafeRLHF harmful + Alpaca benign	none	1000 (100%)	1000 (100%)	1000 (100%)	1000 (100%)
general	Alpaca-cleaned	none	1000 (100%)	1000 (100%)	1000 (100%)	1000 (100%)

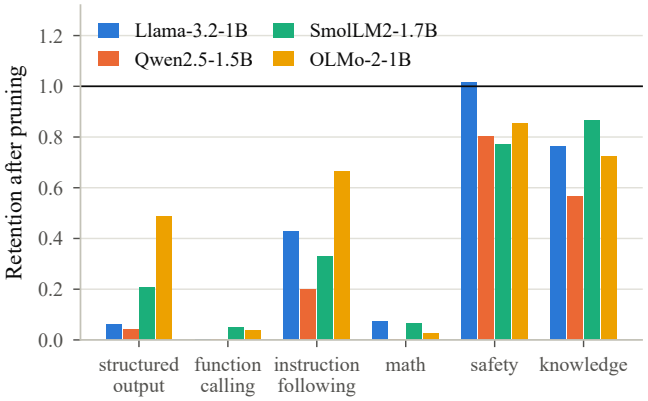


FIGURE 2. Retention after the main cut (test split). Function calling and math break first on every family; safety and knowledge are largely preserved.

0.00–0.05) and math is next (0.00–0.07), while safety refusal (0.77–1.02) and knowledge (0.57–0.87) are largely preserved. Structured output and instruction following sit in between and vary most across families (0.04–0.49 and 0.20–0.66). The mean breakage ranks are 1.4 (function calling), 1.9 (math), 2.8 (structured output), 4.0 (instruction following), 5.3 (knowledge), and 5.8 (safety), and the four rankings agree with $W = 0.92$. The failures of the generative skills are largely structural: after the main cut, Llama produces parseable JSON for 10% of structured-output prompts and 6% of function-calling prompts (99% and 93% before pruning), and Qwen for 16% and 0%. The lighter cut is not uniformly gentler: removing four of Qwen's 28 blocks already removes function calling and math entirely, whereas Llama keeps 75–90% of everything but math at one removed block. Pruning also moves the safety operating point: Llama's over-refusal rises from 0.04 to 0.12 and OLMo's calibration error doubles (0.14 to 0.28).

B. DAMAGE DOES NOT DETERMINE RECOVERABILITY

Table 5 and Figs. 3–5 report the fitted recovery models. Ceilings range from 0 to 1.13 and time-scales from the 3k lower bound of the fit (a recovery faster than the pilots can resolve) to 555k tokens, and the two are not tied to damage in any simple way. On Llama, structured output (damage 0.91) has a ceiling of 0.96 and a time-scale of 13k tokens, so it is fully recoverable with a few tens of thousands of tokens, whereas math (damage 0.93) has a ceiling above 1 but a time-scale of 530k tokens, forty times slower. On SmolLM2, math (damage 0.91) has a ceiling of 0.04: under this recipe it does not come back. On Qwen, safety is above the unpruned model already (ceiling 0) and knowledge gains at most 0.06. On OLMo, instruction following (damage 0.20) did not improve in any pilot (ceiling 0). The rank correlation between damage and ceiling is 0.77 (Llama), 0.67 (Qwen), -0.09 (SmolLM2), and 0.43 (OLMo), none significant with six capabilities; the correlation between damage and time-scale is 0.43, 0.03, -0.14, and -0.77. Damage is therefore a rough guide to *how much* can come back on two families and no guide at all to *how fast*, and the second quantity decides where a finite budget should go.

The transfer matrices (Fig. 5) are mostly small (below 0.3) with two systematic exceptions: knowledge benefits from every pool (for example, instruction and safety data count 0.64 and 0.68 of own-pool tokens on Llama, and instruction data 0.54 on OLMo), because any instruction data improves letter-choice answering on a pruned model; and on SmolLM2 function calling benefits from every pool (0.35–0.48). Negative interference exists in the raw pilot table (on Llama, 100k tokens of math data lowered function calling by 0.07) but is excluded from the fitted model by construction. The fit residuals (0.02–0.19 root-mean-square) reflect the 60-item development evaluations; the largest residuals occur for capabilities whose pilots fluctuate around 1.

C. THE ALLOCATIONS

Fig. 6 shows where each rule sends the budget. Damage-proportional allocation looks similar on every family because the damage profiles are similar (Section VI-A): roughly a quarter each to structured output, function calling, and math, and nothing to safety. RAH differs sharply and family-specifically. On Llama it gives math the trust-region maximum (60%), instruction following 20%, function calling 15%, and structured output only 4%, since structured output saturates after a few tens of thousands of tokens; it also switches to the last-k scope. On Qwen it gives function calling 52% and structured output 24%, and nothing to the safety and general pools whose capabilities have no headroom. On SmolLM2 and OLMo the uncapped objective exposed its failure mode: it sent 60% to instruction following (SmolLM2) and 54% to safety (OLMo) because those capabilities were predicted to exceed the unpruned model, which the capped metric does not reward; the capped objective moves the bud-

TABLE 4. Retention after pruning on the test split, for the light and the main cut, with the absolute unpruned scores and the breakage ranks (1 = most damaged). *W* is Kendall's coefficient of concordance of the four rankings.

Family	Quantity	Struct.	Tool	Instr.	Math	Safety	Knowl.
Llama-3.2-1B	unpruned score	0.32	0.54	0.45	0.47	0.62	0.32
	retention, light cut	0.75	0.89	0.90	0.35	1.27	0.75
	retention, main cut	0.06	0.00	0.43	0.07	1.02	0.76
	breakage rank (1 = worst)	2	1	4	3	6	5
Qwen2.5-1.5B	unpruned score	0.78	0.86	0.38	0.48	0.73	0.58
	retention, light cut	0.11	0.00	0.28	0.01	0.88	0.65
	retention, main cut	0.04	0.00	0.20	0.00	0.80	0.57
	breakage rank (1 = worst)	3	1	4	1	6	5
SmolLM2-1.7B	unpruned score	0.71	0.69	0.45	0.39	0.74	0.43
	retention, light cut	0.52	0.38	0.60	0.12	0.96	0.86
	retention, main cut	0.21	0.05	0.33	0.06	0.77	0.87
	breakage rank (1 = worst)	3	1	4	2	5	6
OLMo-2-1B	unpruned score	0.60	0.54	0.60	0.62	0.85	0.36
	retention, light cut	0.76	0.87	0.86	0.06	0.91	0.99
	retention, main cut	0.49	0.04	0.66	0.02	0.85	0.72
	breakage rank (1 = worst)	3	2	4	1	6	5
All	mean rank ($W = 0.92$)	2.75	1.38	4.00	1.88	5.75	5.25

TABLE 5. Fitted recovery models on the development split: damage, recovery ceiling a_c, time-scale τ_c (thousands of own-pool tokens; 3 is the lower bound of the fit and means faster than the pilots can resolve; n/a when the ceiling is zero), fit residual, and Spearman correlations between damage and the fitted quantities (six capabilities, none significant).

Family	Quantity	Struct.	Tool	Instr.	Math	Safety	Knowl.
Llama-3.2-1B	damage $1 - r_c$	0.91	1.00	0.59	0.93	-0.08	0.48
	ceiling a_c	0.96	0.70	0.18	1.13	0.12	0.41
	time-scale τ_c (k tokens)	13	35	3	530	3	52
	fit RMSE	0.089	0.054	0.045	0.052	0.118	0.121
	ρ(damage, a_c) / ρ(damage, τ_c)	0.77 ($p = 0.07$) / 0.43 ($p = 0.40$)					
Qwen2.5-1.5B	damage $1 - r_c$	0.93	1.00	0.89	1.00	-0.03	0.45
	ceiling a_c	0.67	0.37	0.33	0.10	0.00	0.06
	time-scale τ_c (k tokens)	38	91	24	29	n/a	3
	fit RMSE	0.052	0.016	0.107	0.028	0.189	0.066
	ρ(damage, a_c) / ρ(damage, τ_c)	0.67 ($p = 0.15$) / 0.03 ($p = 0.96$)					
SmolLM2-1.7B	damage $1 - r_c$	0.76	0.91	0.50	0.91	0.29	0.29
	ceiling a_c	0.58	0.66	0.70	0.04	0.39	0.27
	time-scale τ_c (k tokens)	120	90	246	39	56	40
	fit RMSE	0.100	0.079	0.110	0.066	0.075	0.058
	ρ(damage, a_c) / ρ(damage, τ_c)	-0.09 ($p = 0.87$) / -0.14 ($p = 0.79$)					
OLMo-2-1B	damage $1 - r_c$	0.29	0.97	0.20	0.94	0.08	0.50
	ceiling a_c	0.26	0.79	0.00	0.24	0.28	0.35
	time-scale τ_c (k tokens)	37	15	n/a	136	555	29
	fit RMSE	0.123	0.049	0.162	0.035	0.026	0.070
	ρ(damage, a_c) / ρ(damage, τ_c)	0.43 ($p = 0.40$) / -0.77 ($p = 0.07$)					

get to structured output (52%) on SmolLM2 and to math (60%) on OLMo.

D. MAIN COMPARISON

Tables 6 and 7 and Figs. 7 and 8 give the test results with three seeds per method.

Llama-3.2-1B. Capped mean retention is 0.82 (uniform), 0.83 (damage-proportional), 0.85 (RAH), and 0.84 (RAH-capped). RAH beats uniform by $+0.031$ ($p = 0.014$) and damage-proportional by $+0.021$ ($p = 0.075$). The worst-retained capability, math, rises from 0.38 to 0.54 ($+0.17$ versus uniform, $p < 0.001$; $+0.12$ versus damage-proportional, $p = 0.02$), and instruction following from 0.67 to 0.76, at the cost of structured output (1.49 to 1.13 for the capped variant), which was already above the unpruned model. Damage-proportional allocation produces the highest *uncapped* mean (1.07) by pushing structured output to twice its unpruned score; the capped metric discounts this, as intended.

Qwen2.5-1.5B. Capped mean retention is 0.62, 0.61, and 0.64; RAH beats damage-proportional by $+0.024$ ($p = 0.039$) and uniform by $+0.020$ ($p = 0.145$). Worst-case retention is a tie, because math is essentially unrecoverable on this family (ceiling 0.10) under every mixture. The safety operating point differs strongly: the unpruned Qwen refuses 97% of harmful and 51% of benign XSTest prompts, the uniform mixture reproduces this behavior (100% and 59%),

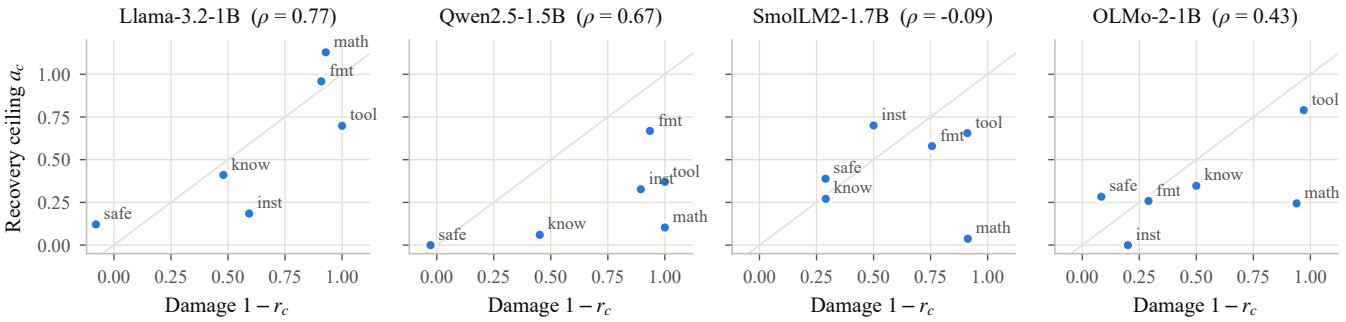


FIGURE 3. **Damage versus fitted recovery ceiling per capability (development split). Points on the diagonal would be fully recoverable; points far below it are damaged but not recoverable under this recipe. ρ is Spearman's correlation over the six capabilities.**

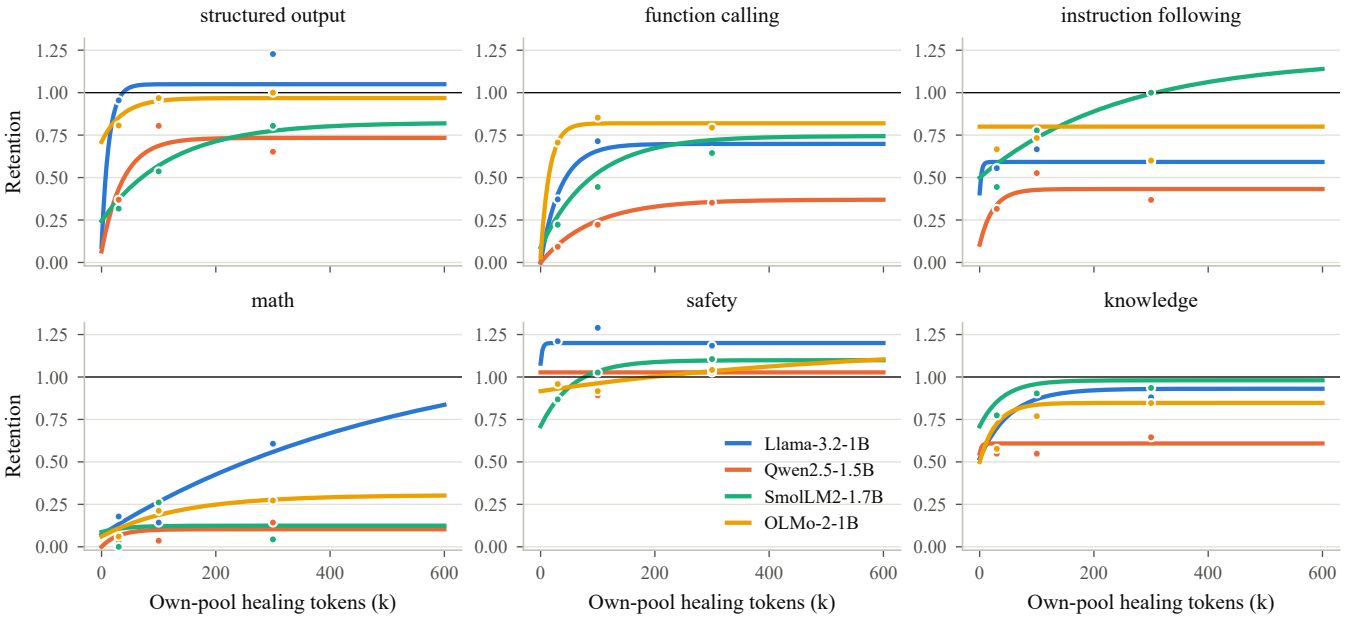


FIGURE 4. **Fitted recovery curves** (7) **per capability with the own-pool pilot observations (dots) at 30k, 100k, and 300k tokens. The same capability recovers at very different speeds and to very different ceilings across families.**

while RAH, which assigns no tokens to the safety pool, refuses 50% of harmful and 6% of benign prompts; the balanced safety score is similar (0.96 versus 0.99 retention). RAH-capped is identical to RAH here.

SmolLM2-1.7B. All four methods tie on capped mean retention (0.76–0.78, $p > 0.29$ for every comparison). RAH and RAH-capped have a slightly lower worst case (0.10 versus 0.15; $p = 0.10$ and $p = 0.045$ against uniform and damage-proportional respectively) because they give math only 4–7% of the budget on the strength of a 0.04 ceiling, whereas the baselines' 17–25% math share still buys a little. RAH-capped lowers over-refusal from 0.21 to 0.08, again by refusing less overall (harmful-prompt refusal 0.43 versus 0.75).

OLMo-2-1B. Damage-proportional allocation has the best capped mean (0.77): on this family the two most damaged capabilities, function calling and math, are also the two most recoverable, so the damage rule happens to coincide with recoverability, while the uncapped RAH objective spent 54% of the budget on safety (0.75). The capped objective repairs this: RAH-capped reaches 0.76, better than uniform ($+0.025$, $p = 0.043$) and tied with damage-proportional (-0.008, $p = 0.44$), and has the best worst case, lifting math from 0.21 to 0.35 ($+0.14$ versus uniform, $p < 0.001$; $+0.04$ versus damage-proportional, $p = 0.11$).

Across the four families, RAH (capped where it differs) is never worse than uniform healing on the primary metric and significantly better on two; against damage-proportional allocation it wins on two families and ties on two. On worst-case retention it gains $+0.17$ and $+0.14$ on Llama and OLMo, ties on Qwen, and loses 0.05 on SmolLM2. Seed-to-seed variation is substantial (structured-output retention on Llama varies by ± 0.5 between seeds of the same method), which is why the comparisons are paired at the item level and why single-seed numbers below are read with caution.

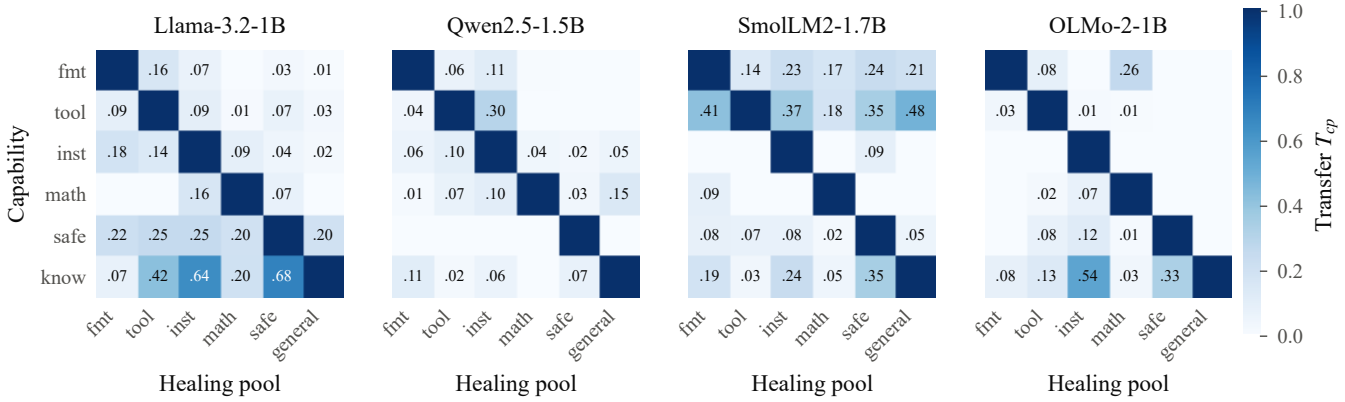


FIGURE 5. Fitted transfer matrices T_{cp}: how many effective tokens one token of pool p contributes to capability c (own pool = 1). Knowledge benefits from every pool; most other transfer is small.

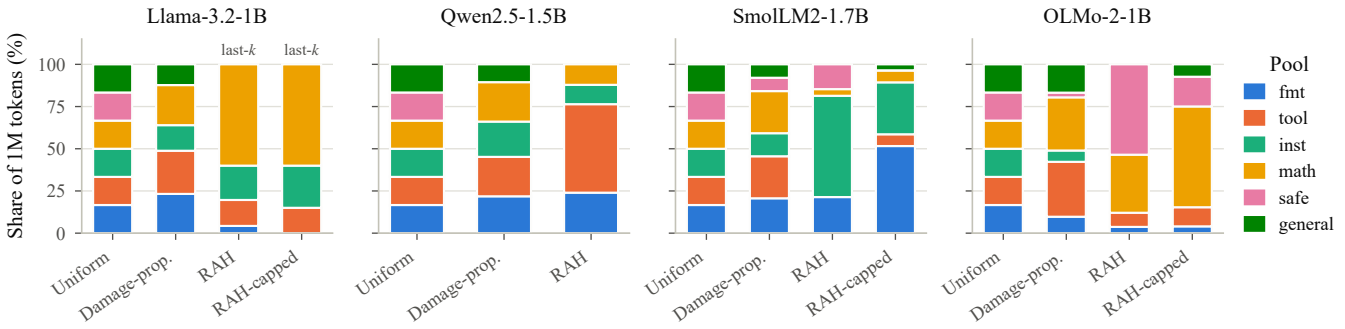


FIGURE 6. Token allocation of the four methods at $B = 1$M (the scope is all-LoRA unless annotated). Damage-proportional allocation is nearly the same on every family; RAH is family-specific. On Qwen, RAH-capped coincides with RAH because the only capability predicted above the unpruned model, safety, has a zero ceiling, so neither objective allocates to it.

E. SCOPE MATTERS, AND DIFFERS BY FAMILY

Table 8 reports the scope pilots. Full fine-tuning of the last three blocks is the best scope on Llama (development capped mean 0.82 versus 0.78 for all-LoRA) and clearly the worst on the other three families (0.43, 0.47, and 0.64 versus 0.57, 0.68, and 0.72). The 1M-token main runs confirm the pilots: last-k with the uniform mixture scores 0.83 versus 0.82 for all-LoRA on Llama, but 0.46 versus 0.62 on Qwen, 0.59 versus 0.77 on SmolLM2, and 0.69 versus 0.74 on OLMo, with worst-case retention of 0.06 on Qwen. The recipe of [13], tuning only the last layers, therefore does not transfer across families at this scale, and a single 300k-token pilot per scope is enough to avoid losses of 0.16, 0.18, and 0.05 in mean retention on Qwen, SmolLM2, and OLMo. The best scope also differs by capability within a family: on Llama, instruction following, math, and safety recover best under last-k, structured output under cut-LoRA, and function calling and knowledge under all-LoRA; on OLMo, math prefers cut-LoRA and safety last-k. A per-capability scope cannot be realized in a single heal, so RAH picks the scope that is best on the sum.

F. ABLATIONS, VARIANTS, AND NEGATIVE RESULTS

Table 9 lists every variant (one seed each except RAH). Removing the transfer matrix changes the mixtures substantially (on Llama, 25% of the budget moves to the general pool) but changes the capped mean by at most 0.02, and on Qwen it is slightly better (0.65 versus 0.64) with a higher worst case. Removing scope selection costs 0.02 on Llama, the only family where RAH chose a scope other than all-LoRA; on the other three families the ablation is the same configuration as RAH's first seed (the OLMo run is bitwise identical), and its differences of at most 0.01 measure run-to-run nondeterminism. The hyperbolic curve gives 0.83, 0.63, 0.77, and 0.77: the curve family matters less than the measured ceilings and timescales. The value of RAH therefore lies mainly in the pilots themselves, with transfer and curve shape second-order.

The max–min objective does what it is designed to do and shows what it costs. It gives the least recoverable capability the trust-region maximum (60% on math on every family) and raises the worst case above uniform healing on all four families (0.51, 0.27, 0.22, and 0.31 versus 0.38, 0.21, 0.15, and 0.21) and above RAH on three, but the capped mean falls below uniform on three families (0.83, 0.60, 0.75, and 0.61 versus 0.82, 0.62, 0.77, and 0.74), and on OLMo its scope rule chose cut-LoRA, which lost function calling (0.17). With one seed these numbers are fragile, but the trade-off is the expected one: the sum objective with the cap is the better default unless the worst capability is all that matters. Two variants are negative results. The pilot-free proxy produces near-

TABLE 6. Per-capability retention on the test split (three seeds; per-item scores averaged over seeds) and the headline metrics. Bold marks the best capped mean and worst case per family. Over-refusal is the fraction of benign XSTest prompts refused; ECE is the expected calibration error on MMLU.

Family	Method	Struct.	Tool	Instr.	Math	Safety	Knowl.	Capped mean	Mean	Worst	Over-refusal	ECE
Llama-3.2-1B	Uniform	1.49	0.88	0.67	0.38	1.27	1.15	0.82	0.97	0.38	0.16	0.18
	Damage-prop.	2.02	0.90	0.66	0.43	1.34	1.09	0.83	1.07	0.43	0.15	0.19
	RAH	1.49	0.82	0.76	0.54	1.27	1.09	**0.85**	1.00	0.54	0.09	0.26
	RAH-capped	1.13	0.77	0.74	0.55	1.28	1.11	0.84	0.93	**0.55**	0.09	0.27
Qwen2.5-1.5B	Uniform	0.73	0.54	0.58	0.21	0.96	0.67	0.62	0.62	**0.21**	0.59	0.26
	Damage-prop.	0.73	0.54	0.56	0.19	1.09	0.66	0.61	0.63	0.19	0.09	0.28
	RAH	0.77	0.58	0.62	0.19	0.99	0.68	**0.64**	0.64	0.19	0.06	0.24
	RAH-capped	identical allocation to RAH on this family (see Fig. 6)										
SmolLM2-1.7B	Uniform	0.90	0.93	0.70	0.15	1.04	0.96	0.77	0.78	**0.15**	0.21	0.20
	Damage-prop.	0.87	0.92	0.68	0.14	1.04	0.96	0.76	0.77	0.14	0.19	0.21
	RAH	0.88	0.96	0.78	0.10	1.06	0.91	0.77	0.78	0.10	0.19	0.26
	RAH-capped	0.96	0.93	0.79	0.10	0.91	0.97	**0.78**	0.78	0.10	0.08	0.21
OLMo-2-1B	Uniform	0.77	0.97	0.71	0.21	0.96	0.81	0.74	0.74	0.21	0.16	0.26
	Damage-prop.	0.84	0.99	0.73	0.31	0.96	0.80	**0.77**	0.77	0.31	0.12	0.26
	RAH	0.77	0.97	0.67	0.29	0.97	0.83	0.75	0.75	0.29	0.15	0.32
	RAH-capped	0.78	0.98	0.66	0.35	0.98	0.83	0.76	0.76	**0.35**	0.16	0.23

TABLE 7. Item-level paired bootstrap (2,000 resamples, seeds pooled) on the pre-registered metrics: differences, 95% confidence intervals, and two-sided *p*-values.

Family	Comparison	Δ capped mean [95% CI]	*p*	Δ worst case [95% CI]	*p*
Llama-3.2-1B	RAH vs Uniform	+0.031 [+0.006, +0.053]	0.014	+0.168 [+0.070, +0.267]	<0.001
	RAH vs Damage-prop.	+0.021 [-0.003, +0.045]	0.075	+0.116 [+0.021, +0.207]	0.020
	RAH-capped vs Uniform	+0.021 [-0.006, +0.044]	0.123	+0.172 [+0.081, +0.270]	<0.001
	RAH-capped vs Damage-prop.	+0.011 [-0.016, +0.034]	0.436	+0.119 [+0.028, +0.211]	0.012
Qwen2.5-1.5B	RAH vs Uniform	+0.020 [-0.007, +0.045]	0.145	-0.027 [-0.086, +0.031]	0.369
	RAH vs Damage-prop.	+0.024 [+0.001, +0.042]	0.039	-0.007 [-0.069, +0.052]	0.863
SmolLM2-1.7B	RAH vs Uniform	-0.001 [-0.023, +0.022]	0.953	-0.048 [-0.104, +0.009]	0.101
	RAH vs Damage-prop.	+0.008 [-0.015, +0.036]	0.453	-0.043 [-0.095, +0.009]	0.116
	RAH-capped vs Uniform	+0.003 [-0.021, +0.027]	0.883	-0.052 [-0.104, +0.004]	0.076
	RAH-capped vs Damage-prop.	+0.013 [-0.012, +0.037]	0.296	-0.048 [-0.095, -0.004]	0.045
OLMo-2-1B	RAH vs Uniform	+0.013 [-0.009, +0.034]	0.251	+0.075 [+0.027, +0.129]	0.002
	RAH vs Damage-prop.	-0.020 [-0.039, +0.001]	0.057	-0.027 [-0.067, +0.011]	0.176
	RAH-capped vs Uniform	+0.025 [+0.001, +0.048]	0.043	+0.142 [+0.083, +0.210]	<0.001
	RAH-capped vs Damage-prop.	-0.008 [-0.029, +0.012]	0.436	+0.040 [-0.008, +0.089]	0.107

uniform allocations on every family, because the cross-family linear map turns the pruned-versus-teacher likelihood gap into nearly constant ceilings, and consequently scores within 0.015 of uniform healing everywhere, including Qwen's 0.63 over-refusal. The training-free linear patch reaches a capped mean of only 0.37–0.65 and a worst case of 0.07–0.13: a linear map at the cut restores some structure but no generative capability at this scale.

G. BUDGET SWEEP

Fig. 9 shows the Qwen sweep. At 0.25M tokens RAH is below uniform (0.52 versus 0.55 mean retention): the fitted mixture (38% function calling, 35% structured output, 21% instruction following, 6% math, nothing for the safety and general pools) loses more on the capabilities it starves than it gains on the ones it targets at this budget. At 1M it is above (0.64 versus 0.62, three seeds). At 4M the trust region (600k per pool, relaxed to $B/|\mathcal{P}|$) forces the uniform allocation, so the two runs train on the same mixture with the same seed; their difference of 0.003 is a measurement of run-to-run nondeterminism rather than of the method. The sweep therefore says that the pilot-fitted allocation pays off at moderate budgets and must not be trusted far below the budgets at which the curves were measured. Read as a compute-matched comparison, the 4M uniform run also exceeds RAH at 1M (capped mean 0.66 versus 0.64); Section VII-B discusses this.

H. SIDE EFFECTS: REFUSAL AND CALIBRATION

Over-refusal is governed by the share of the safety pool and by the teacher's own behavior. The unpruned Llama refuses 28% of harmful and 4% of benign prompts; every healed Llama refuses more of both (66–81% and 9–16%), and RAH, which allocates nothing to the safety pool, refuses the least. The unpruned Qwen refuses 97% of harmful and 51% of benign prompts, so uniform healing, which includes the safety pool, faithfully restores an over-refusing teacher. The balanced XSTest score moves little across methods (0.91–1.09 retention); what the allocation changes is the operating point. Calibration moves in the other direction: healing lowers the ECE of pruned Llama from 0.48 to 0.17–0.19 under

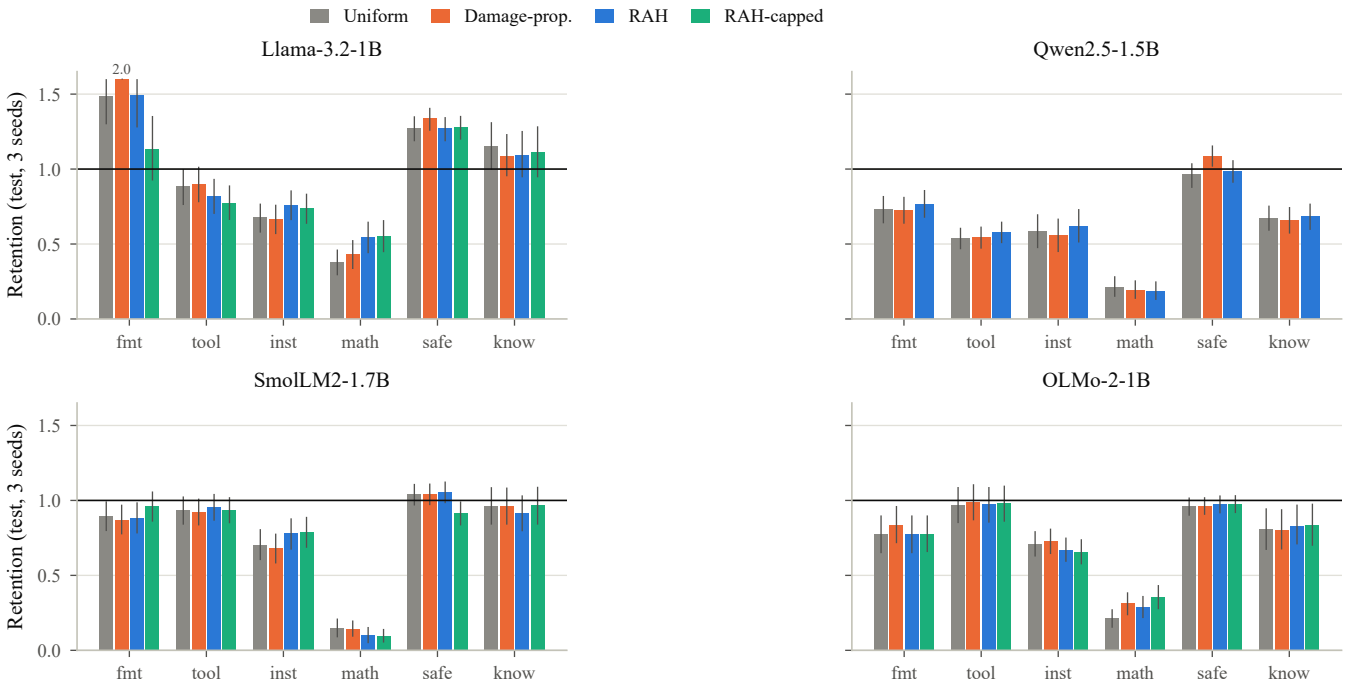


FIGURE 7. Retention by method and capability on the test split (three seeds; whiskers are 95% bootstrap intervals over items). Values above 1 exceed the unpruned model; bars clipped at the axis top carry their value.

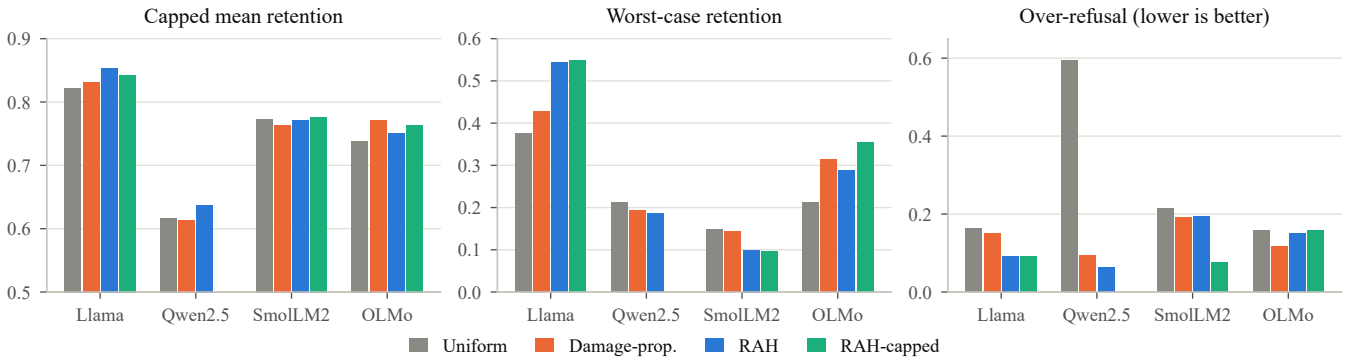


FIGURE 8. Headline metrics by family. RAH is never below uniform healing on capped mean retention and raises the worst case strongly on Llama and OLMo; over-refusal moves with the share of the safety pool in the mixture.

uniform and damage-proportional allocation but only to 0.26–0.27 under RAH, whose last-k full fine-tuning and math-heavy mixture increase confidence; on OLMo the uncapped RAH has the highest ECE (0.32). A calibration term in the objective would be a natural extension.

VII. DISCUSSION

A. WHEN RECOVERABILITY-AWARE ALLOCATION HELPS

RAH helps when the recoverability profile departs from the damage profile: on Llama, where a cheap-to-restore capability (structured output) and an expensive one (math) are equally damaged, and on Qwen, where function calling has headroom and math has none. It ties when every mixture saturates the recoverable capabilities (SmolLM2) or when the damage rule coincidentally matches recoverability (OLMo, where it took the capped objective to match damage-proportional allocation). It also removes a hidden choice from the baselines: the uniform mixture's fixed share of safety data moved Qwen to a 59% over-refusal operating point, whereas RAH allocates safety data only where the pilots show headroom. These results support the premise behind PASER [7], that recovery data should be allocated by capability; what they question is damage as the allocation signal, and the evidence for that is suggestive rather than decisive, since no comparison against the damage rule survives a correction for multiple testing.

B. WHAT THE PILOTS COST

The comparisons of Section VI hold the healing budget fixed at 1M tokens for every method, which is the question a practi-

TABLE 8. Scope pilots (300k tokens, uniform mixture, development split) and the scope chosen by RAH, with the 1M-token test check of last-k against all-LoRA under the uniform mixture.

Family	Scope	Struct.	Tool	Instr.	Math	Safety	Knowl.	Capped mean	Worst
Llama-3.2-1B	all-LoRA	1.45	0.86	0.56	0.29	1.03	1.08	0.78	0.29
	cut-LoRA	1.50	0.71	0.63	0.18	1.00	0.92	0.74	0.18
	last-k (chosen)	1.36	0.80	0.81	0.32	1.05	1.00	0.82	0.32
	1M check: last-k / all-LoRA, uniform mix	capped mean 0.83 / 0.82, worst 0.49 / 0.38 (test split)							
Qwen2.5-1.5B	all-LoRA (chosen)	0.76	0.41	0.42	0.11	1.03	0.74	0.57	0.11
	cut-LoRA	0.65	0.22	0.37	0.04	0.95	0.65	0.48	0.04
	last-k	0.46	0.11	0.42	0.07	0.95	0.58	0.43	0.07
	1M check: last-k / all-LoRA, uniform mix	capped mean 0.46 / 0.62, worst 0.06 / 0.21 (test split)							
SmolLM2-1.7B	all-LoRA (chosen)	0.66	0.71	0.67	0.04	1.00	1.06	0.68	0.04
	cut-LoRA	0.46	0.60	0.50	0.00	0.97	0.84	0.56	0.00
	last-k	0.39	0.36	0.28	0.04	1.08	0.74	0.47	0.04
	1M check: last-k / all-LoRA, uniform mix	capped mean 0.59 / 0.77, worst 0.06 / 0.15 (test split)							
OLMo-2-1B	all-LoRA (chosen)	0.84	0.97	0.70	0.09	0.90	0.85	0.72	0.09
	cut-LoRA	0.81	0.88	0.67	0.15	0.94	0.69	0.69	0.15
	last-k	0.81	0.68	0.70	0.03	0.98	0.65	0.64	0.03
	1M check: last-k / all-LoRA, uniform mix	capped mean 0.69 / 0.74, worst 0.17 / 0.21 (test split)							

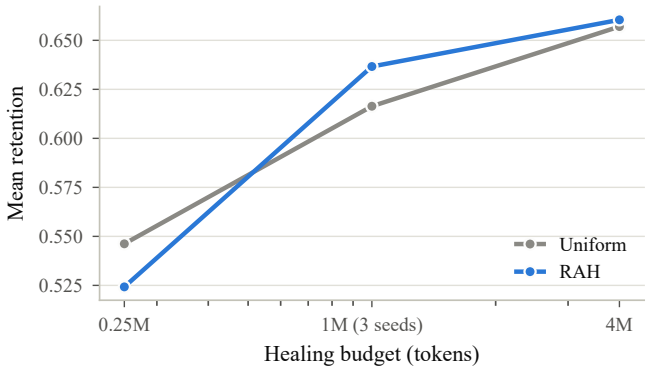


FIGURE 9. Budget sweep on Qwen2.5-1.5B (mean retention; one seed at 0.25M and 4M, three seeds at 1M). At 4M the trust region makes the two allocations identical.

tioner with a fixed budget asks. They do not hold total training compute fixed: RAH's pilots process 3.48M tokens before the heal, so spending the same total on a longer uniform heal is a legitimate alternative. The one such comparison available cuts against RAH: on Qwen2.5-1.5B, uniform healing with 4M tokens reaches a capped mean retention of 0.66 against 0.64 for RAH at 1M (Fig. 9). The pilots therefore pay for themselves in two situations: when the heal itself is budget-limited, by memory, time, or a token cap imposed by a deployment pipeline, and when the fitted recovery model is reused, since re-planning for a different budget or different priority weights is a millisecond solve and the pilots double as a diagnosis of which capabilities can come back at all. For a single heal with no token limit, a longer uniform heal may simply be the better use of the compute. A compute-matched sweep on all four families is the obvious next experiment; we did not run it.

C. A PRACTITIONER'S RECIPE

Run the 21 pilots (3.5 times the tokens of one heal, each evaluated on 60 items per capability), fit the model with the τ lower bound and the ridge term, use the capped objective, keep the trust region, and choose the scope from the scope pilots. The fitted model is reusable: a new budget, new deployment weights w_c, or a different cap changes the solve, not the pilots. The decision to run pilots is the cost–benefit one of Section VII-B: at this scale they gain 0.02–0.03 in capped mean and up to 0.17 in worst-case retention at a fixed budget, which is worth 3.5 heals when the pruned model will be deployed widely or re-healed for several budgets, and not worth it for a one-off model with a generous budget.

D. THREATS TO VALIDITY AND LIMITATIONS

Scale. All families are between 1.2B and 1.7B parameters; whether the profiles persist at 3B–8B is untested. *Evaluation size.* Test subsets of 150–300 items give confidence intervals of about ± 0.1 per capability, and development sets of 60 items make the pilots noisy (fit residuals up to 0.19). *Single pruning criterion and recipe.* Block Influence and teacher-regenerated targets are held fixed; the ceilings are properties of this recipe, and a stronger healing recipe may raise them. *Three budget points per pool.* Ceilings beyond 300k tokens are extrapolated; the trust region limits but does not remove this risk, and the 0.25M sweep shows the fitted allocation can mislead far below the pilot budgets. *Refusal classifier.* A string-match classifier over-counts polite non-refusals and under-counts indirect ones; the over-refusal numbers should be read as operating points, not precise rates. *Seeds.* Variants other than the four core methods use one seed. *Hardware.* Two GPU types were used; the L4 seeds used checksum-identical data, and the 4M-token repeat quantifies run-to-run nondeterminism at 0.003 mean retention. *Compute.* The baselines are matched on healing tokens, not on total compute (Section VII-B). *Language.* All data are English.

TABLE 9. Ablations, variants, and the training-free baseline (one seed each; RAH with three seeds for reference). The mixture lists the percentage of the budget given to each pool.

Family	Variant	Scope	Mixture % (fmt/tool/inst/math/safe/general)	Capped mean	Worst	Over-refusal
Llama-3.2-1B	RAH (3 seeds)	last-k	4 / 15 / 20 / 60 / 0 / 0	0.85	0.54	0.09
	RAH, no transfer	last-k	7 / 11 / 6 / 50 / 1 / 25	0.83	0.54	0.09
	RAH, no scope choice	all-LoRA	4 / 15 / 20 / 60 / 0 / 0	0.83	0.52	0.19
	RAH, hyperbolic curve	last-k	13 / 27 / 0 / 60 / 0 / 0	0.83	0.56	0.12
	RAH-maxmin	last-k	8 / 8 / 8 / 60 / 8 / 8	0.83	0.51	0.17
	RAH-proxy	all-LoRA	21 / 18 / 18 / 14 / 11 / 18	0.82	0.37	0.13
	Last-k + uniform	last-k	17 / 17 / 17 / 17 / 17 / 17	0.83	0.49	0.16
	Linear patch	–	–	0.65	0.13	0.04
Qwen2.5-1.5B	RAH (3 seeds)	all-LoRA	24 / 52 / 11 / 12 / 0 / 0	0.64	0.19	0.06
	RAH, no transfer	all-LoRA	14 / 24 / 9 / 53 / 0 / 0	0.65	0.27	0.06
	RAH, no scope choice	all-LoRA	24 / 52 / 11 / 12 / 0 / 0	0.63	0.15	0.02
	RAH, hyperbolic curve	all-LoRA	34 / 51 / 5 / 10 / 0 / 0	0.63	0.16	0.05
	RAH-maxmin	all-LoRA	8 / 8 / 8 / 60 / 8 / 8	0.60	0.27	0.56
	RAH-proxy	all-LoRA	17 / 17 / 17 / 17 / 17 / 17	0.63	0.25	0.63
	Last-k + uniform	last-k	17 / 17 / 17 / 17 / 17 / 17	0.46	0.06	0.71
	Linear patch	–	–	0.37	0.07	0.33
SmolLM2-1.7B	RAH (3 seeds)	all-LoRA	21 / 0 / 60 / 4 / 15 / 0	0.77	0.10	0.19
	RAH, no transfer	all-LoRA	29 / 20 / 29 / 0 / 12 / 9	0.77	0.08	0.18
	RAH, no scope choice	all-LoRA	21 / 0 / 60 / 4 / 15 / 0	0.78	0.14	0.21
	RAH, hyperbolic curve	all-LoRA	25 / 0 / 54 / 0 / 21 / 0	0.77	0.09	0.26
	RAH-maxmin	all-LoRA	8 / 8 / 8 / 60 / 8 / 8	0.75	0.22	0.24
	RAH-proxy	all-LoRA	17 / 17 / 17 / 17 / 16 / 16	0.76	0.13	0.23
	Last-k + uniform	last-k	17 / 17 / 17 / 17 / 17 / 17	0.59	0.06	0.32
	Linear patch	–	–	0.49	0.08	0.25
OLMo-2-1B	RAH (3 seeds)	all-LoRA	4 / 8 / 0 / 34 / 54 / 0	0.75	0.29	0.15
	RAH, no transfer	all-LoRA	17 / 8 / 0 / 25 / 27 / 23	0.75	0.30	0.16
	RAH, no scope choice	all-LoRA	4 / 8 / 0 / 34 / 54 / 0	0.74	0.27	0.16
	RAH, hyperbolic curve	all-LoRA	2 / 14 / 0 / 44 / 40 / 1	0.77	0.37	0.16
	RAH-maxmin	cut-LoRA	0 / 0 / 40 / 60 / 0 / 0	0.61	0.31	0.03
	RAH-proxy	all-LoRA	17 / 17 / 16 / 17 / 15 / 17	0.75	0.25	0.17
	Last-k + uniform	last-k	17 / 17 / 17 / 17 / 17 / 17	0.69	0.17	0.11
	Linear patch	–	–	0.61	0.09	0.06

VIII. CONCLUSION

Depth pruning of sub-2B instruction-tuned models breaks function calling and math first and safety and knowledge last, consistently across four families, but how much and how fast each capability comes back is family-specific and only loosely related to how badly it broke. Recoverability-Aware Healing measures recoverability with cheap pilot heals and turns the measurement into an allocation of healing data and adaptation scope by a concave program with explicit safeguards. At an equal healing budget it is never worse than uniform healing on the pre-registered metric, improves the worst-retained capability by up to 0.17, and matches or beats damage-proportional allocation on every family; the scope choice alone avoids losses of up to 0.18 on three of four families. The pilots cost 3.5 times the tokens of one heal, and when that compute is spent on a longer uniform heal instead the advantage can disappear, so RAH is a tool for budget-limited or repeated healing rather than a free improvement. A pilot-free proxy did not work and is reported as such; the max–min objective buys worst-case retention at the price of the mean. Future work includes replication at 3B–8B, other pruning criteria, on-policy distillation targets inside the same allocation framework, a calibration term in the objective, and learned proxies that could replace the pilots.

ACKNOWLEDGMENT

The authors thank the providers of the open checkpoints and datasets used in this study. Portions of the text of this article were drafted with the assistance of an AI language model (Claude, Anthropic) under the authors' direction, and the experimental code was developed with AI assistance; the authors reviewed and verified all content, experiments, and results. Computation used two laptop GPUs, a Google Colab A100, and a Lightning AI L4 instance.

REFERENCES

[1] X. Men, M. Xu, Q. Zhang, Q. Yuan, B. Wang, H. Lin, Y. Lu, X. Han, and W. Chen, "ShortGPT: Layers in large language models are more redundant than you expect," in *Findings Assoc. Comput. Linguist.: ACL 2025*, Jul. 2025, pp. 20 192–20 204, arXiv:2403.03853.

[2] A. Gromov, K. Tirumala, H. Shapourian, P. Glorioso, and D. A. Roberts, "The unreasonable ineffectiveness of the deeper layers," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025, arXiv:2403.17887.

[3] B.-K. Kim, G. Kim, T.-H. Kim, T. Castells, S. Choi, J. Shin, and H.-K. Song, "Shortened LLaMA: A simple depth pruning for large language models," in *Proc. ICLR Workshop Math. Empir. Understanding Found. Models (ME-FoMo)*, 2024, arXiv:2402.02834.

[4] J. Song, K. Oh, T. Kim, H. Kim, Y. Kim, and J.-J. Kim, "SLEB: Streamlining LLMs through redundancy verification and elimination of transformer blocks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 235, 2024, pp. 46 136–46 155, arXiv:2402.09025.

[5] S. Muralidharan, S. T. Sreenivas, R. Joshi, M. Chochowski, M. Patwary, M. Shoeybi, B. Catanzaro, J. Kautz, and P. Molchanov, "Compact language

models via pruning and knowledge distillation," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, arXiv:2407.14679.

[6] V. Thangarasa, G. Venkatesh, M. Lasby, N. Sinnadurai, and S. Lie, "Self-data distillation for recovering quality in pruned large language models," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 7, 2025, arXiv:2410.09982.

[7] B. He, L. Yin, H.-L. Zhen, X. Zhang, M. Yuan, and C. Ma, "PASER: Post-training data selection for efficient pruned large language model recovery," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026, arXiv:2502.12594.

[8] Y. Li, Z. Liu, and E. Xing, "Data mixing optimization for supervised fine-tuning of large language models," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 267, 2025, pp. 35 419–35 437, arXiv:2508.11953.

[9] H. Zhang, "DomainPilot: Domain-level loss-guided two-stage data mixture optimization for efficient language model fine-tuning," arXiv preprint arXiv:2607.22769, 2026.

[10] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, arXiv:2305.14314.

[11] X. Chen, Y. Hu, J. Zhang, Y. Wang, C. Li, and H. Chen, "Streamlining redundant layers to compress large language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025, arXiv:2403.19135.

[12] S. T. Sreenivas, S. Muralidharan, R. Joshi, M. Chochowski, A. S. Mahabaleshwarkar, G. Shen, J. Zeng, Z. Chen, Y. Suhara, S. Diao, C. Yu, W.-C. Chen *et al.*, "LLM pruning and distillation in practice: The Minitron approach," arXiv preprint arXiv:2408.11796, 2024.

[13] Y. Lu, H. Cheng, Y. Fang, Z. Wang, J. Wei, D. Xu, Q. Xuan, X. Yang, and Z. Zhu, "Reassessing layer pruning in LLMs: New insights and methods," arXiv preprint arXiv:2411.15558, 2024.

[14] Y. Lu, Y. Li, W. Xie, S. Yu, Q. Xuan, Z. Zhu, and S. Wen, "The structural scalpel: Automated contiguous layer pruning for large language models," arXiv preprint arXiv:2510.23652, 2025.

[15] L. G. Mugnaini, C. T. Duarte, A. H. R. Costa, and A. Jordao, "Layer pruning with consensus: A triple-win solution," *IEEE Access*, vol. 13, pp. 148 192–148 202, 2025.

[16] E. Frantar and D. Alistarh, "SparseGPT: Massive language models can be accurately pruned in one-shot," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 202, 2023, pp. 10 323–10 337, arXiv:2301.00774.

[17] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, "A simple and effective pruning approach for large language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024, arXiv:2306.11695.

[18] X. Ma, G. Fang, and X. Wang, "LLM-Pruner: On the structural pruning of large language models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, arXiv:2305.11627.

[19] M. Xia, T. Gao, Z. Zeng, and D. Chen, "Sheared LLaMA: Accelerating language model pre-training via structured pruning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024, arXiv:2310.06694.

[20] X. Chen, H. Bai, T. Yuan, R. Liu, K. Zhao, X. Yu, L. Hou, T. Guan, Y. He, and C. Yuan, "A simple linear patch revives layer-pruned large language models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 38, 2025, arXiv:2505.24680.

[21] V.-D. Yun, J. Jo, S. P. Karimireddy, and S. Lee, "Ghosted layers: Unconstrained activation alignment for recovering layer-pruned LLMs," arXiv preprint arXiv:2605.15491, 2026.

[22] Z. Feng, H. Zhou, Z. Zhu, T. Li, J. J. D. Chua, L. O. Mak, G. W. Ng, and K. Mao, "Restoring pruned large language models via lost component compensation," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 38, 2025, arXiv:2510.21834.

[23] Q. Zhang, Q. Yuan, H. Lin, Y. Lu, X. Han, L. Sun, M. Xu, and J. Li, "Short-OPD: Recovering pruned LLMs with short-to-long on-policy distillation," arXiv preprint arXiv:2607.13124, 2026.

[24] A. Jaiswal, Z. Gan, X. Du, B. Zhang, Z. Wang, and Y. Yang, "Compressing LLMs: The truth is rarely pure and never simple," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024, arXiv:2310.01382.

[25] S. M. Xie, H. Pham, X. Dong, N. Du, H. Liu, Y. Lu, P. Liang, Q. V. Le, T. Ma, and A. W. Yu, "DoReMi: Optimizing data mixtures speeds up language model pretraining," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, arXiv:2305.10429.

[26] C. Zhou, S. Han, L. Yang, Y. Zhou, X. Cheng, Y. Wang, and H. Li, "RankAdaptor: Hierarchical rank allocation for efficient fine-tuning pruned LLMs via performance model," in *Findings Assoc. Comput. Linguist.: NAACL 2025*, Apr. 2025, pp. 5796–5810, arXiv:2406.15734.

[27] M. Wagner, C. Roux, M. Zimmer, and S. Pokutta, "A free lunch in LLM compression: Revisiting retraining after pruning," arXiv preprint arXiv:2510.14444, 2025.

[28] M. K. K. Siam, M. Tausif-Ul-Islam, M. R. R. Khan, M. A. Hossain, M. Amin, L. H. Khan, N. Farhan, and F. Sadeque, "Exploring the limits of pruning: Task-specific neurons, model collapse, and recovery in task-specific large language models," arXiv preprint arXiv:2604.27115, 2026.

[29] S. Shrestha, A. Shrestha, A. Nepal, M. Kim, and K. Ross, "On the limits of layer pruning for generative reasoning in large language models," arXiv preprint arXiv:2602.01997, 2026.

[30] C. Zhang, V. Patil, H. Lange, and U. Aleem, "What breaks under pruning in smart homes, and when? Evaluating LLM degradation across architectures and task complexity," arXiv preprint arXiv:2609.17515, 2026.

[31] P. Martra, "Fragile knowledge, robust instruction-following: The width pruning dichotomy in Llama-3.2," arXiv preprint arXiv:2512.22671, 2025.

[32] A. Kharinaev, V. Moskvoretskii, E. Shvetsov, K. Studenikina, M. Bykov, and E. Burnaev, "Investigating the impact of quantization methods on the safety and reliability of large language models," *IEEE Access*, vol. 14, pp. 96 771–96 793, 2026.

[33] Z. Xu, A. Gupta, T. Li, O. Bentham, and V. Srikumar, "Beyond perplexity: Multi-dimensional safety evaluation of LLM compression," in *Findings Assoc. Comput. Linguist.: EMNLP 2024*, Nov. 2024, pp. 15 359–15 396, arXiv:2407.04965.

[34] V. K. Chhabra and M. M. Khalili, "Towards understanding and improving refusal in compressed models via mechanistic interpretability," arXiv preprint arXiv:2504.04215, 2025.

[35] S. S. Alahmari, L. O. Hall, P. R. Mouton, and D. B. Goldgof, "Repeatability of fine-tuning large language models illustrated using QLoRA," *IEEE Access*, vol. 12, pp. 153 221–153 231, 2024.

[36] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, arXiv:1706.03762.

[37] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022, arXiv:2106.09685.

[38] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan *et al.*, "The Llama 3 herd of models," arXiv preprint arXiv:2407.21783, 2024.

[39] Qwen, A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin *et al.*, "Qwen2.5 technical report," arXiv preprint arXiv:2412.15115, 2024.

[40] L. Ben Allal, A. Lozhkov, E. Bakouch, G. Martín Blázquez, G. Penedo, L. Tunstall, A. Marafioti, H. Kydlíček, A. Piqueres Lajarín, V. Srivastav, J. Lochner, C. Fahlgren *et al.*, "SmolLM2: When smol goes big – data-centric training of a small language model," arXiv preprint arXiv:2502.02737, 2025.

[41] Team OLMo, P. Walsh, L. Soldaini, D. Groeneveld, K. Lo, S. Arora, A. Bhagia, Y. Gu, S. Huang, M. Jordan, N. Lambert, D. Schwenk *et al.*, "2 OLMo 2 furious," arXiv preprint arXiv:2501.00656, 2025, shorter version accepted at COLM 2025.

[42] S. Geng, H. Cooper, M. Moskal, S. Jenkins, J. Berman, N. Ranchin, R. West, E. Horvitz, and H. Nori, "JSONSchemaBench: A rigorous benchmark of structured outputs for language models," arXiv preprint arXiv:2501.10868, 2025.

[43] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica, and J. E. Gonzalez, "The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 267, 2025, pp. 48 371–48 392.

[44] J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou, "Instruction-following evaluation for large language models," arXiv preprint arXiv:2311.07911, 2023.

[45] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac'h, H. Li, K. McDonell *et al.*, "A framework for few-shot language model evaluation," [Online]. Available: https://zenodo.org/records/10256836, 2023, zenodo, v0.4.0, Dec. 2023. Accessed: Oct. 7, 2026.

[46] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," arXiv preprint arXiv:2110.14168, 2021.

[47] P. Röttger, H. Kirk, B. Vidgen, G. Attanasio, F. Bianchi, and D. Hovy, "XSTest: A test suite for identifying exaggerated safety behaviours in large language models," in *Proc. Conf. North Amer. Chapter Assoc. Comput.*

Linguist.: Hum. Lang. Technol. (NAACL-HLT), Vol. 1 (Long Papers), 2024, pp. 5377–5400, arXiv:2308.01263.
[48] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021, arXiv:2009.03300.
[49] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 70, 2017, pp. 1321–1330, arXiv:1706.04599.
[50] Glaive AI, "glaive-function-calling-v2 (hugging face dataset)," [Online]. Available: https://huggingface.co/datasets/glaiveai/glaive-function-calling-v2, 2023, accessed: Oct. 7, 2026.
[51] yahma, "Alpaca-Cleaned: A cleaned version of the Stanford Alpaca dataset (hugging face dataset)," [Online]. Available: https://huggingface.co/datasets/yahma/alpaca-cleaned, 2023, accessed: Oct. 7, 2026.
[52] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, "Stanford Alpaca: An instruction-following LLaMA model," [Online]. Available: https://github.com/tatsu-lab/stanford_alpaca, 2023, accessed: Oct. 7, 2026.
[53] J. Ji, D. Hong, B. Zhang, B. Chen, J. Dai, B. Zheng, T. A. Qiu, J. Zhou, K. Wang, B. Li, S. Han, Y. Guo, and Y. Yang, "PKU-SafeRLHF: Towards multi-level safety alignment for LLMs with human preference," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguist. (ACL), Vol. 1 (Long Papers)*, Jul. 2025, pp. 31 983–32 016, arXiv:2406.15513.
[54] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul, B. Bossan, and M. Tietz, "PEFT: State-of-the-art parameter-efficient fine-tuning methods," [Online]. Available: https://github.com/huggingface/peft, 2022, accessed: Oct. 7, 2026.
[55] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. Conf. Empir. Methods Nat. Lang. Process.: Syst. Demonstrations (EMNLP)*, 2020, pp. 38–45.

PIYUSH DHOKA TODO (100–150 words): education, current position with location, research interests.

VEDANT PANDHARE TODO (100–150 words).

VARUN INAMDAR TODO (100–150 words).

AADARSH PATHRE TODO (100–150 words).

PAVITHA NOOJI TODO (100–150 words).

SIXTH AUTHOR TODO: placeholder for the sixth author's biography (100–150 words).